Week 05

함수 · 모듈 · 테스트 가능한 코드

AI활용코딩연습(ZBE135)

이영호 교수 · 국립목포대학교 컴퓨터학부

지난주 돌아보기

```
groups = {}
for dept, score in pairs:
    groups.setdefault(dept, []).append(score)

top3 = sorted(rows, key=lambda r: (-r["total"], r["name"]))[:3]
kept = [s for s in scores if s >= 60]     # 반복 중 삭제 대신
```

> 원본은 바꾸지 않고 **새 결과를 돌려준다** → 오늘 배울 "순수 함수"의 핵심

오늘의 목표

1. 매개변수 · 기본값 · 키워드 인자 · docstring으로 **읽기 쉬운 함수**를 만든다
2. `print` 대신 `return`, 전역 대신 매개변수 → **순수 함수**
3. `test_` 함수와 `assert`로 테스트하고, 잘못된 입력은 `raise`
4. `%%writefile`로 **모듈**을 만들어 `import`한다
5. AI 코드의 **가변 기본값 버그**를 찾는다

수업 흐름

구간	시간	활동
도입 · 개념	20분	함수 설계, 순수 함수, 테스트
안내 실습	45분	함수 구성 · 범위 · 테스트 · raise
독립 · 짝 실습	35분	성적 계산 모듈 만들기
공유 · 점검	20분	가변 기본값 버그, 과제 안내

이번 주 Colab 실습 열기

왜 함수로 나누는가?

함수 없이	함수로 나누면
같은 계산을 여러 곳에 복사	한 곳만 고치면 됨
어디가 틀렸는지 찾기 어려움	함수 단위로 테스트
코드가 길어 읽기 힘듦	이름이 곧 설명
AI에게 전체를 맡김	작은 함수 단위로 요청 · 검증

> AI에게도 **작은 함수 하나씩** 요청하면 검증하기 쉽다

함수의 구성 요소

```
def weighted_total(midterm: float, final: float, w_mid: float = 0.4) -> float:
    """중간.기말 점수의 가중 합계를 돌려준다. 기말 가중치는 1 - w_mid."""
    w_final = 1 - w_mid
    return midterm * w_mid + final * w_final

weighted_total(80, 90)                  # 기본값 사용
weighted_total(80, 90, 0.5)             # 위치 인자
weighted_total(final=90, midterm=80)    # 키워드 인자
help(weighted_total)                    # docstring이 설명서
```

여러 값 돌려주기

```
def describe(scores):
    """점수 리스트의 (평균, 최저, 최고)를 돌려준다."""
    avg = sum(scores) / len(scores)
    return round(avg, 1), min(scores), max(scores)

avg, low, high = describe([72, 95, 88, 59])
```

`return a, b, c` → **튜플 하나**를 돌려주고, 받는 쪽에서 언패킹

예상해 보기: print vs return

```
def show_total(midterm, final):
    print("합계:", midterm + final)

result = show_total(80, 90)
print(result)          # ?
```

정답: 화면에 `합계: 170`이 나오고, `result`는 **None**

- `print`는 보여 줄 뿐, 값을 넘겨주지 않는다
- 계산 함수는 **return**, 출력은 함수 **밖**에서

지역 변수와 전역 변수

```
bonus = 5
def add_bonus(score):
    result = score + bonus      # 전역을 읽는 건 OK
    return result
# print(result)                 # NameError: 지역 변수

counter = 0
def increase():
    counter = counter + 1       # UnboundLocalError
```

전역에 기대는 함수 → **테스트하기 어렵다**

순수 함수

순수 함수	부수효과가 있는 함수
같은 입력 → 항상 같은 출력	전역 · 파일 · 화면에 따라 달라짐
입력을 바꾸지 않음	입력 리스트를 수정
`assert f(2) == 4`로 바로 테스트	테스트 전에 상태 준비 필요

```
def increase_pure(counter):
    return counter + 1
```

> 필요한 건 **매개변수로 받고**, 결과는 **return으로**

잘못된 입력은 raise로 알린다

```
def letter_grade(total):
    if not 0 <= total <= 100:
        raise ValueError(f"총점은 0~100이어야 합니다: {total}")
    if total >= 90:
        return "A"
    ...

letter_grade(105)
# ValueError: 총점은 0~100이어야 합니다: 105
```

조용히 엉뚱한 값을 내는 것보다 **즉시 멈추는 것**이 낫다

(받아서 처리하는 `try/except` 는 6주차)

테스트 함수

```
def test_letter_grade():
    assert letter_grade(95) == "A"          # 정상
    assert letter_grade(90) == "A"          # 경계
    assert letter_grade(89.99) == "B"       # 바로 아래
    assert letter_grade(0) == "F"           # 범위 끝
    for bad in [-1, 100.1]:                 # 잘못된 입력
        try:
            letter_grade(bad)
            assert False, f"{bad}에서 ValueError가 나야 함"
        except ValueError:
            pass
```

코드를 고칠 때마다 **한 번에 다시 확인**

작은 테스트 실행기

```
def run_tests(namespace):
    for name, obj in list(namespace.items()):
        if name.startswith("test_") and callable(obj):
            try:
                obj()
                print(f"✅ {name}")
            except AssertionError as e:
                print(f"❌ {name}: {e}")

run_tests(globals())
```

실무에서는 **pytest**가 같은 일을 한다 (`test_` 함수를 자동으로 찾아 실행)

실습 1 · 2 (Level 1 → Level 2)

Level 1 따라하기

- 기본값 · 키워드 인자 · 여러 값 반환 · `help()`
- `UnboundLocalError` 재현, 순수 함수로 바꾸기
- `raise ValueError`, 테스트 함수, 테스트 실행기

Level 2 바꾸기

- `weighted_total`에 과제 점수 · 가중치 추가
- **기존 호출 결과가 그대로**인지 테스트로 확인 (하위 호환)

모듈: 함수를 파일로

```
%%writefile grade_utils.py
"""성적 계산 함수 모음."""
WEIGHTS = {"midterm": 0.3, "final": 0.4, "assignment": 0.3}

def weighted_total(midterm, final, assignment, weights=None): ...
def letter_grade(total): ...
def attendance_ok(attended, total_sessions=16, minimum=12): ...
def final_grade(record): ...

if __name__ == "__main__":
    print(final_grade({...}))     # 직접 실행할 때만
```

모듈 불러오기

```
import importlib
import grade_utils
importlib.reload(grade_utils)      # 파일을 고친 뒤 다시 불러오기

from grade_utils import final_grade, weighted_total as wt

grade_utils.WEIGHTS
final_grade({"midterm": 80, "final": 90, "assignment": 85, "attendance": 15})
# (85.5, 'B')
```

방식	사용
`import 모듈`	`모듈.함수()`
`from 모듈 import 함수`	`함수()`
`import 모듈 as 별명`	`별명.함수()`

실습 3 (Level 3): 성적 계산 모듈

성적 규정 (가상)

- 총점 = 중간 30% + 기말 40% + 과제 30%
- 출석 16회 중 12회 미만이면 **F**
- 과제 미제출(`None`)은 0점

```
def final_grade(record):
    total = weighted_total(record["midterm"], record["final"], record.get("assignment"))
    if not attendance_ok(record["attendance"]):
        return total, "F"
    return total, letter_grade(total)
```

작은 함수를 **조합**하면 규정이 코드에 그대로 드러난다

모듈 테스트

```
def test_module_final_grade():
    assert final_grade({"midterm": 80, "final": 90,
                        "assignment": 85, "attendance": 15}) == (85.5, "B")
    assert final_grade({"midterm": 95, "final": 98,
                        "assignment": 100, "attendance": 11}) == (97.7, "F")
    assert final_grade({"midterm": 95, "final": 98,
                        "assignment": 100, "attendance": 12})[1] == "A"
```

출석 경계 **11 / 12**, 과제 **None**, 가중치 합 오류까지

AI 코드: 가변 기본값

"목록을 안 주면 새 목록을 만들어서 추가하는 함수"

```
def add_record_ai(record, records=[]):
    records.append(record)
    return records

class_a = add_record_ai({"name": "김민준"})
class_b = add_record_ai({"name": "이서연"})
print(class_b)    # [{'name': '김민준'}, {'name': '이서연'}]  ← ?!
```

기본값 `[]`는 **정의할 때 한 번만** 만들어져 계속 재사용된다

해결: 기본값은 None

```
def add_record(record, records=None):
    if records is None:
        records = []
    records.append(record)
    return records

def test_add_record_independent():
    a = add_record({"name": "김민준"})
    b = add_record({"name": "이서연"})
    assert a is not b
```

AI에게 함수를 요청할 때

```
- print 하지 말고 return 할 것
- 입력 리스트나 딕셔너리를 수정하지 말 것
- 기본값으로 [] 나 {} 를 쓰지 말 것
- 잘못된 입력에는 ValueError
- 정상·경계·잘못된 입력을 포함한 test_ 함수를 함께 작성
```

> AI가 쓴 테스트는 **AI 코드의 실수를 그대로 닮을 수 있다**
> → 내가 생각한 경계값 테스트를 **하나 이상 추가**

흔한 실수 모음

실수	결과	해결
계산 함수에서 `print` 만	`None` 반환	`return`
함수 안에서 전역 수정	`UnboundLocalError`	매개변수 · 반환값
`def f(x, items=[])`	호출 간 데이터 공유	`items=None`
모듈 수정 후 그대로 사용	옛 코드 실행	`importlib.reload`
테스트를 AI에게만 맡김	같은 실수 반복	내 경계값 테스트 추가

이번 주 과제 (10점)

문제	내용	배점
1	`weighted_average` + 잘못된 입력 `ValueError` 4가지	2
2	`password_strength` 비밀번호 강도 (도우미 함수로 나누기)	2
3	`text_stats` 글자 · 단어 · 문장 · 평균 단어 길이	3
4	**테스트로 AI 버그 잡기**: `my_tests` 작성 + `median` 구현	3

문제 4: 내 테스트가 **정답은 통과**, **버그 함수 2개는 잡아내야** 만점

자기 점검

1. `print`만 하는 함수가 테스트하기 어려운 이유는?
2. 순수 함수의 두 가지 조건은?
3. 좋은 테스트에 들어가야 할 입력 세 종류는?
4. `importlib.reload`가 필요한 상황은?
5. `def f(x, items=[])`의 문제와 해결책은?

정리와 다음 주

오늘	핵심
함수	기본값 · 키워드 인자 · 여러 값 반환 · docstring
설계	return, 매개변수, 입력 불변 → 순수 함수
검증	`raise ValueError`, `test_` 함수, 테스트 실행기
모듈	`%%writefile`, `import`, `reload`, `__main__`
AI	가변 기본값, 내 경계값 테스트 추가

다음 주 · Week 06 파일 · CSV · 예외 처리 — 오류에 견고한 CSV 성적 처리